Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Кафедра прикладної математики

Звіт

про виконання лабораторної роботи №1

з курсу «Основи інженерії даних»

Виконала:
студентка групи ПМІД-11
Мазур Д. М.

Перевірив:
Алєксєєв В. І.

Львів — 2026

Тема: Зберігання та доступ до даних для аналітики. Порівняння рядкового (CSV) та стовпчастого (Parquet) форматів зберігання.

Мета роботи: Зрозуміти практичний вплив різних форматів файлів на ефективність зберігання даних та продуктивність простих запитів. Дослідити алгоритми стиснення, а також стовпчастий і рядковий формати зберігання на високому рівні.

Постановка задачі: Переробити великий CSV-файл з даними фінансових транзакцій в оптимізований формат для аналітичних запитів.

Завдання до роботи:

1. Згенерувати великий набір даних фінансових транзакцій і зберегти його у форматі CSV.
2. Перетворити дані CSV у формат Parquet із різними алгоритмами стиснення (без стиснення, Snappy, GZIP).
3. Порівняти розміри отриманих файлів.
4. Виконати прості аналітичні запити до кожного файлу та порівняти продуктивність.

Хід виконання роботи:

1. **Генерація даних.** Скрипт `lab1_db/main.py` генерує 1 000 000 транзакцій за допомогою NumPy з фіксованим зерном (`seed = 42`), що робить експеримент відтворюваним. Схему даних наведено в табл. 1. Запис у CSV тривав 1.36 с. Після запису DataFrame видаляється з пам'яті, і далі програма працює лише з файлом, як у реальній задачі.

Табл. 1: Схема набору даних транзакцій

Поле	Тип	Значення
`transaction_id`	рядок	`TXN_00000000` … `TXN_00999999`
`user_id`	ціле	1000–9998
`amount`	дійсне	5.00–5000.00
`category`	рядок	5 категорій (Groceries, Travel, …)
`status`	рядок	COMPLETED, PENDING, FAILED, REFUNDED
`timestamp`	дата/час	від 2025-01-01, крок 1 с

2. **Конвертація CSV у Parquet.** CSV-файл зчитується з розпізнаванням дат і записується у Parquet трьома способами за допомогою рушія PyArrow:

```
parquet_codecs = {"Parquet (None)": None,
                  "Parquet (Snappy)": "snappy",
                  "Parquet (GZIP)": "gzip"}
for name, codec in parquet_codecs.items():
    csv_df.to_parquet(parquet_paths[name], compression=codec, index=False)
```

Час запису: без стиснення 0.18 с, Snappy 0.15 с, GZIP 0.86 с. GZIP записує файл майже у 6 разів довше, бо його алгоритм стиснення значно складніший.

3. **Порівняння розмірів файлів.** Результати наведено в табл. 2 та на рис. 1.

Табл. 2: Розміри файлів для 1 000 000 рядків

Формат	Розмір, MB	Зменшення відносно CSV
CSV (без стиснення)	61.59	–
Parquet (None)	33.42	45.7%
Parquet (Snappy)	18.65	69.7%
Parquet (GZIP)	12.16	80.3%

Навіть без стиснення Parquet майже вдвічі менший за CSV. Числа зберігаються у бінарному вигляді, а не як текст, а стовпці з малою кількістю різних значень (`category`, `status`) кодуються словником (dictionary encoding). Стиснення додатково ефективне саме у стовпчастому форматі, бо однотипні значення одного стовпця лежать поруч.

4. **Аналітичні запити.** Для порівняння продуктивності реалізовано два запити:

   - *Запит 1 (агрегація):* сума `amount` за кожною категорією (`GROUP BY category`);
   - *Запит 2 (фільтрація):* кількість та середня сума транзакцій зі статусом COMPLETED і сумою понад 1000.

   Для CSV читаються лише потрібні стовпці (`usecols`), проте файл однаково доводиться розбирати повністю. Для Parquet застосовано дві оптимізації стовпчастого формату: *column projection* (з диска читаються лише потрібні стовпці) та *predicate pushdown* (фільтри передаються рушію PyArrow і застосовуються під час читання):

```
df = pd.read_parquet(path, columns=["status", "amount"],
        filters=[("status", "==", "COMPLETED"), ("amount", ">", 1000)])
```

5. **Перевірка коректності.** Перед вимірюваннями програма перевіряє, що кожен Parquet-файл дає такий самий результат, як і CSV (функція `results_equal`). Для всіх 6 комбінацій (2 запити $\times$ 3 файли) отримано збіг.

6. **Вимірювання продуктивності.** Кожен запит спочатку виконується один раз для прогріву (ініціалізація модулів, кеш файлової системи), після чого вимірюється середній час 5 запусків (табл. 3, рис. 1).

Табл. 3: Час виконання запитів (середнє з 5 запусків)

Формат	Запит 1, с	Прискорення	Запит 2, с	Прискорення
CSV	0.1749	1.00x	0.1746	1.00x
Parquet (None)	0.0494	3.54x	0.0164	10.67x
Parquet (Snappy)	0.0492	3.55x	0.0165	10.57x
Parquet (GZIP)	0.0503	3.48x	0.0176	9.95x

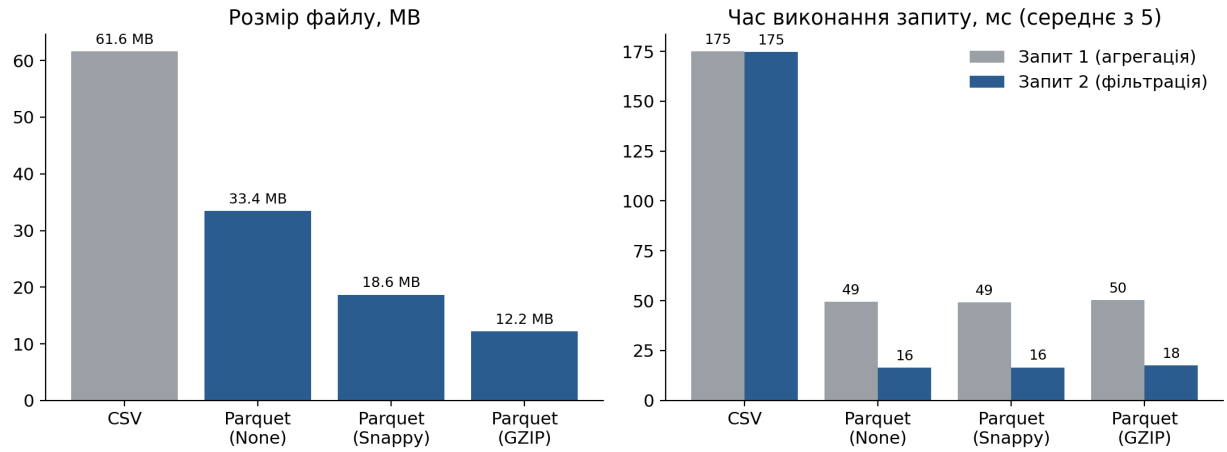


Рис. 1: Розміри файлів та час виконання запитів для різних форматів

Висновки: У ході виконання лабораторної роботи №1 набір з 1 000 000 транзакцій було перетворено з CSV у Parquet з трьома варіантами стиснення. Parquet без стиснення на 45.7% менший за CSV, зі Snappy на 69.7% менший, з GZIP на 80.3% менший. GZIP дає найменший файл, але запис у ньому найповільніший (0.86 с проти 0.15 с у Snappy). Тому для «гарячих» даних, які часто перезаписуються, доцільний Snappy, а для архівного зберігання GZIP. Аналітичні запити до Parquet виконувалися в 3.5 раза швидше для агрегації та приблизно в 10 разів швидше для фільтрації. Це пояснюється стовпчастою організацією: читаються лише потрібні стовпці, дані не потрібно розбирати з тексту, а фільтр застосовується під час читання. Вибір кодека майже не вплинув на швидкість запитів: витрати на розпакування малі порівняно з виграшем від меншого обсягу вводу-виводу. Отже, для аналітичних навантажень стовпчастий формат зі стисненням суттєво ефективніший за рядковий CSV і за обсягом зберігання, і за швидкістю.